User Stories — World Action Models for Cross-Embodiment Robot Control

Team members: Krrish Thakku Suresh, Joshua Jerin

Advisor: Huu Tri Nguyen

Stakeholder Map

Category	Stakeholder	Why they matter
Primary	Robotics researcher adapting the model to a new robot	Directly trains, fine-tunes, and runs the model on a new embodiment
Secondary	Faculty advisor	Judges whether results actually show cross-embodiment generalization
Secondary	Lab manager responsible for robot hardware	Maintains the physical robot and pays the cost if it is damaged
Hidden	Lab members working near the robot during physical tests	Never touch the software, but are physically exposed to its mistakes
Hidden	People who pass through the camera's view during data collection	Can end up in training data without ever knowing the project exists
Hidden	Future students reusing our checkpoints and datasets	Only show up after the project ends, when they try to reproduce or extend it

User Stories

US-01 (primary): As a robotics researcher adding a new robot arm to the lab, I want to adapt the pretrained world action model to that arm using only a small set of demonstrations, so that I can get a working control policy without collecting thousands of new demonstrations for every robot.

US-02 (secondary): As the faculty advisor reviewing the team's results, I want to compare the model's task success rate on robots it was trained on against robots it has never seen, alongside a single-embodiment baseline, so that I can tell whether a performance gain reflects real cross-embodiment generalization rather than overfitting to one robot.

US-03 (hidden): As a lab member working near the robot during physical testing, I want the robot's motion to stay inside a fixed workspace and speed limit no matter what action the model predicts, so that a wrong prediction cannot cause the robot to strike me.

US-04 (hidden): As a lab manager responsible for robot hardware, I want an independent motion-limit layer to reject or clamp unsafe model commands before they reach the robot controller, so that an unreliable checkpoint cannot damage the robot or lab equipment.

US-05 (hidden): As a person who may pass through the camera view during data collection, I want identifiable imagery to be excluded, anonymized, or access-restricted before data is retained, so that I am not unknowingly included in a robotics training dataset.

INVEST Self-Check

Story	Independent	Negotiable	Valuable	Estimable	Small	Testable
US-01	Yes — depends only on having a pretrained model	Yes — number of demonstrations and adaptation method are open	Yes — core purpose of the project	Mostly — depends on which pretrained model we choose	Largest story; if it grows, split into "collect demonstrations" and "fine-tune and evaluate"	Yes — success rate on the new arm after N demonstrations
US-02	Yes — evaluation can run on any checkpoint	Yes — which tasks and baseline are open	Yes — decides whether results are meaningful	Yes	Yes	Yes — two numeric success rates plus a baseline
US-03	Yes — gate works regardless of model details	Yes — pass threshold is set with the lab manager	Yes — protects hardware	Yes	Yes	Yes — attempt to run an unevaluated checkpoint and observe that it is refused
US-04	Yes — enforced below the model, independent of US-03	Yes — exact limits chosen with the lab	Yes — personal safety	Yes	Yes	Yes — send out-of-range actions and measure that commanded motion stays within limits
US-05	Yes — applies to the data pipeline only	Yes — method (drop, blur, restrict recording area) is open	Yes — privacy and compliance	Yes	Yes	Yes — inspect stored frames for identifiable faces

No story names a UI element. Each story states a need and a benefit, and the specific mechanism is left for design in Weeks 5–7.

Stories considered and cut

- *"As a researcher, I want a dashboard showing training curves"* — cut because it names a UI element rather than a need; the underlying need (judging results) is covered by US-02.
- *"As a researcher, I want the model to work on any robot"* — cut because it is not Small or Testable; narrowed into US-01.
- *"As the team, I want to use the GPU cluster"* — cut because it has no stakeholder benefit and is a project constraint, not a requirement.

Use Cases

Numeric thresholds below (demonstration count, success rate, speed and timing limits) are draft targets to be confirmed with our advisor and the lab, and revised in Week 9.

UC-01: Adapt model to a new robot embodiment

Expands: US-01

Primary actor: Robotics researcher

Secondary actors: Simulation environment, training compute (GPU node)

Preconditions

1. A pretrained world action model checkpoint exists and loads without error.

2. The new arm has an embodiment description (joint count, joint limits, action dimension) that loads in the simulator.

3. At least 50 demonstrations of the target task have been recorded on the new arm.

4. An evaluation task set for the new arm is defined in simulation.

Main success flow

1. Researcher selects the pretrained checkpoint and registers the new arm's embodiment description.

2. System validates the description in the simulator and reports the arm's joint count, joint limits, and action dimension.

3. Researcher provides the demonstration set for the target task.

4. System checks every demonstration against the registered action dimension and reports how many were accepted.

5. Researcher starts adaptation.

6. System fine-tunes the model and saves a new checkpoint linked to the demonstration set, training configuration, and embodiment description.

7. Researcher requests an evaluation of the new checkpoint in simulation.

8. System runs 50 evaluation episodes and reports the task success rate next to the single-embodiment baseline.

9. Researcher reviews the results and marks the checkpoint as a hardware candidate.

10. System records the checkpoint as simulation-evaluated, which is required before any physical run (US-03).

Alternate flow A1 — success rate below target (from step 8)

1. System reports a success rate below 60%.

2. System does not allow the checkpoint to be marked as a hardware candidate.

3. Researcher records additional demonstrations and returns to step 3.

Exception flow E1 — demonstrations do not match the embodiment (from step 4)

1. System finds one or more demonstrations whose action dimension differs from the registered arm.

2. System rejects the demonstration set before any training starts and lists each mismatched file with its expected and actual action dimension.

3. No checkpoint is created. Researcher corrects or removes the listed files and returns to step 3.

Postcondition

A new checkpoint exists that is linked to the exact demonstrations, configuration, and embodiment it was trained on, and has a recorded simulation success rate. It is marked as a hardware candidate only if that success rate is at least 60%.

UC-02: Run a physical trial within motion limits

Expands: US-04

Primary actor: Researcher running the physical trial

Secondary actors: Robot controller, lab members present near the robot (protected stakeholder)

Preconditions

1. The checkpoint being tested is marked simulation-evaluated (UC-01, step 10).

2. A workspace boundary and a maximum end-effector speed of 0.25 m/s are configured in the motion-limit layer.

3. The emergency stop has been tested in the current session and stops the robot.

Main success flow

1. Researcher starts a physical trial with the selected checkpoint.

2. System loads the workspace boundary and speed limit and confirms both are active before enabling the robot.

3. Model predicts an action from the current camera observation and robot state.

4. System checks the action against the limits and sends it to the robot controller.

5. Steps 3–4 repeat until the task finishes or the researcher ends the trial.

6. System stops the robot and saves a log of every predicted action, every commanded action, and any limit events.

Alternate flow A1 — predicted action exceeds a limit (from step 4)

1. System finds that the predicted action would leave the workspace or exceed 0.25 m/s.

2. System clamps the action to the nearest value inside the limits, sends the clamped action, and logs the event.

3. Flow continues at step 3.

Exception flow E1 — motion-limit layer stops responding (during steps 3–5)

1. The motion-limit layer stops sending heartbeats to the robot controller.

2. Robot controller halts all motion and ignores further model commands.

3. System logs the failure time. The trial cannot resume until the researcher restarts the limit layer and repeats step 2.

Postcondition

The robot is stopped, all commanded motion during the trial stayed within the configured limits, and a complete trial log is saved.

Acceptance Criteria

UC-01

AC-01.1 (main flow) Given a loadable pretrained checkpoint and 50 valid demonstrations for the new arm, When the researcher runs adaptation and then simulation evaluation, Then a new checkpoint is saved with references to its demonstration set, configuration, and embodiment description, and a report shows the success rate over 50 episodes alongside the baseline success rate.

AC-01.2 (main flow) Given an adapted checkpoint whose simulation success rate is at least 60% over 50 episodes, When the researcher marks it as a hardware candidate, Then the checkpoint's status changes to simulation-evaluated.

AC-01.3 (alternate flow A1) Given an adapted checkpoint whose simulation success rate is below 60%, When the researcher tries to mark it as a hardware candidate, Then the request is refused and the checkpoint's status remains not evaluated.

AC-01.4 (exception flow E1) Given a demonstration set in which at least one file's action dimension differs from the registered arm, When the researcher starts adaptation, Then zero training steps run, no new checkpoint is created, and every mismatched file is listed with its expected and actual action dimension.

UC-02

AC-02.1 (main flow) Given a configured workspace boundary and a 0.25 m/s speed limit, When a physical trial of at least 100 control steps is completed, Then 100% of commanded positions in the trial log are inside the workspace boundary and the maximum logged end-effector speed is at most 0.25 m/s.

AC-02.2 (alternate flow A1) Given a running trial, When the model predicts an action outside the workspace boundary, Then the commanded action sent to the robot is inside the boundary and a limit event is written to the log for that step.

AC-02.3 (exception flow E1) Given a running trial, When the motion-limit layer stops sending heartbeats, Then robot motion stops within 100 ms and no further model command reaches the robot until the limit layer is restarted.

World Action Models for Cross Embodiment Robot Control

D0 High Level System Design | Joshua Jerin and Krrish Thakku Suresh
Advisor: Huu Tri Nguyen | CS 5001 | October 2, 2026

1 Title, goal statement, and conventions

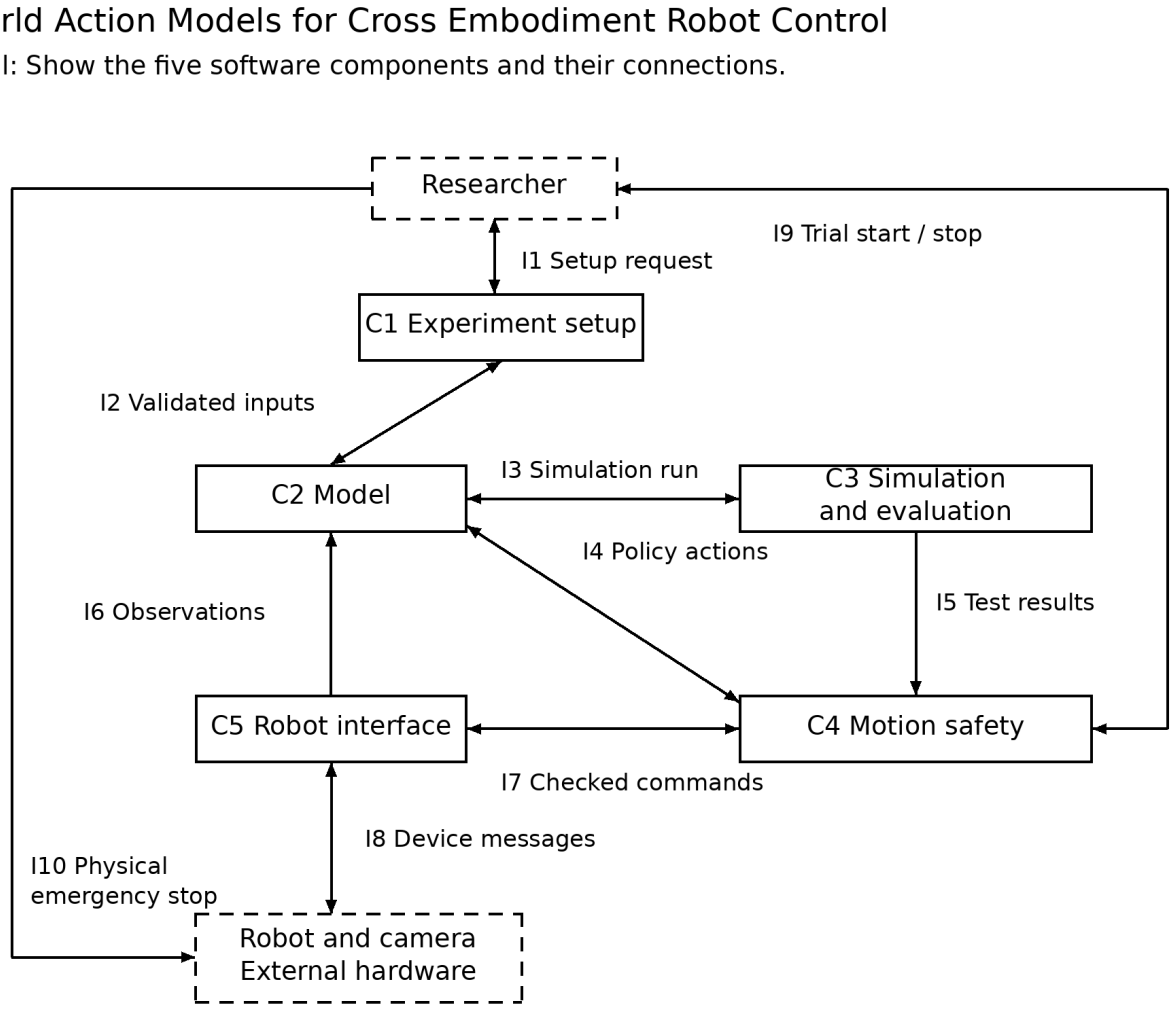
Our goal is to adapt a pretrained world action model to a new robot arm with a small set of demonstrations, compare its performance across robots, and test it safely in a controlled lab setting.

Inputs are a pretrained model, demonstrations, a robot description, and live camera/robot-state data. Outputs are an adapted model, simulation results, checked robot commands, and a trial log.

This is a proposed design. The robot, model, simulator, and numerical limits still need team and advisor approval. C means component. I means interface: a connection between two parts. For example, I9 is the trial start/stop request. Names beside the IDs match the connection table in Section 4.

2 Block diagram (the D0 diagram)

Figure 1. System structure



Legend: Solid boxes = project software; dashed boxes = researcher or hardware.
Arrows = data or control; two arrowheads = both directions.
The physical emergency stop bypasses the software.

3 Component responsibility table

The owner column is a suggested division for the team to confirm. Each component saves its own outputs as ordinary project files; a separate storage service is unnecessary for this first version.

Component	Responsibility	Connections in / out	Proposed owner
C1 Experiment setup	Validate the researcher's dataset and robot setup before starting an experiment.	In/out: I1, I2	Joshua
C2 Model	Adapt the pretrained model or use a saved model to predict actions.	In: I2, I3, I4, I6 Out: I2, I3, I4	Krrish
C3 Simulation and evaluation	Test the model in simulation and compare its success rate with a single-embodiment baseline.	In/out: I3 Out: I5	Krrish
C4 Motion safety	Check that a physical run is allowed and keep each command within the approved limits.	In: I4, I5, I7, I9 Out: I4, I7, I9	Joshua
C5 Robot interface	Translate observations and checked commands between our software and the robot.	In: I7, I8 Out: I6, I7, I8	Krrish

Connection to the submitted user stories

US-01, adapt with few demonstrations: C1 checks the data and embodiment; C2 fine-tunes the model and saves its dataset/configuration references.

US-02, compare seen and unseen robots with a single-embodiment baseline: C3 runs matched tests using C2 and returns the report through C1. The report separates robots used in training from held-out robots.

US-03, protect people nearby: C4 checks workspace and speed limits; C5 sends only checked commands. The lab must keep people outside the approved motion area and provide a working emergency stop.

US-04, protect hardware: C4 rejects or limits unsafe actions before C5 sends them. C3 supplies the simulation evidence needed before a physical run.

US-05, protect camera privacy: C1 excludes or anonymizes identifiable images before normal dataset storage; C5 keeps live images temporary unless they go through that review.

This also covers every component: C1 supports US-01/02/05; C2 supports US-01/02; C3 supports US-01/02/04; C4 supports US-03/04; C5 supports US-03/04/05.

A few operating rules

Before training, C1 rejects demonstrations with the wrong action dimension; C2 asks C3 to load and check the robot description through I3. C2 saves the adapted checkpoint with the dataset, robot description, settings and code version. C3 saves the test report. C4 saves the predicted action, sent action and any limit event for each physical step.

C4 runs separately from the model process. C5 is the only software component allowed to send robot commands. C5 forwards C4's heartbeat unchanged to the controller, whose independent timeout must stop the robot if it disappears. Verify this on the actual hardware before a physical trial.

4 Interface specification table

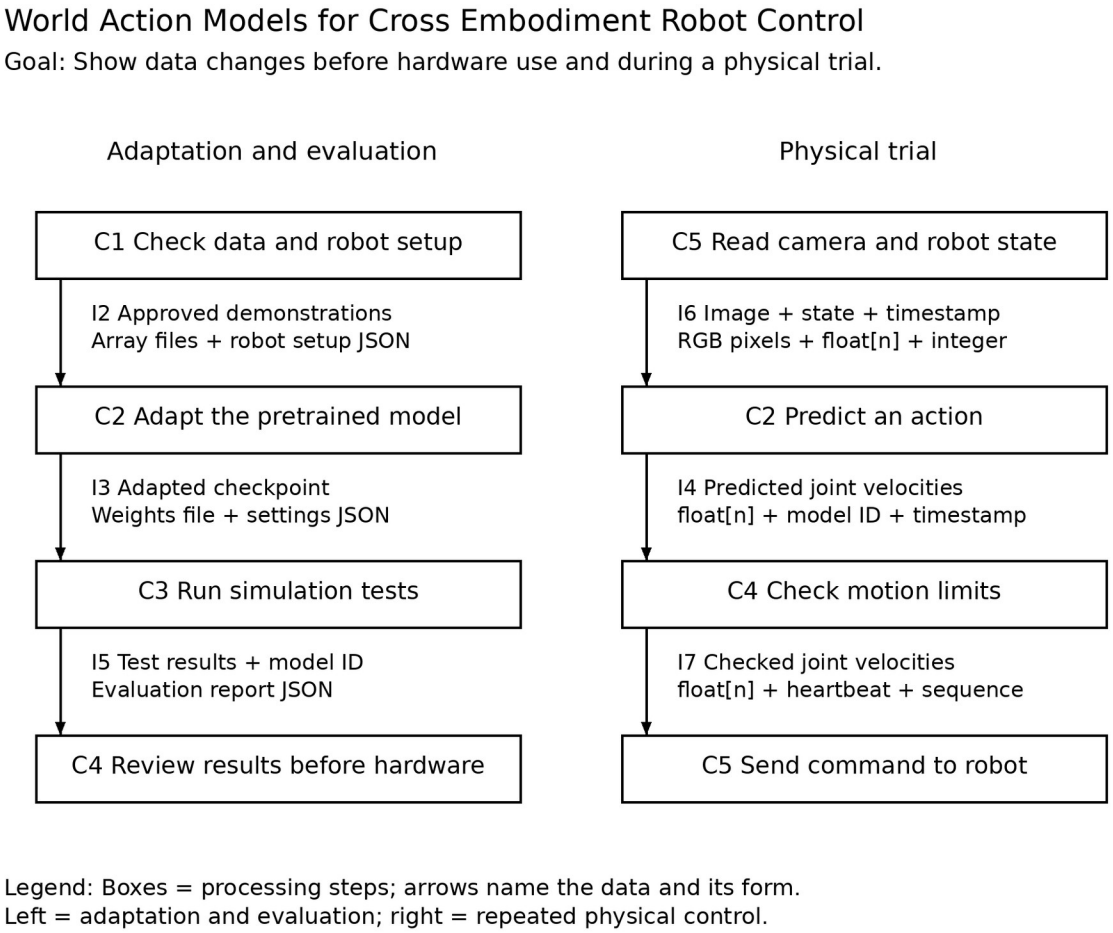
I1–I10 match Figure 1. JSON is used for small configuration/status records; images, arrays and model weights remain files or memory buffers. Here, n is the registered joint count. float[n] means n finite numbers in the agreed joint order. Joint positions use radians and joint velocities use radians per second. IDs/paths are strings; counts and monotonic timestamps (ms) are integers; success rates are 0–1 floats. Status is an enum: ready, running, complete, failed or stopped. RGB pixels are uint8[H,W,3]; image size, joint count and motion limits come from robot/model setup JSON. A heartbeat is an increasing integer. Local process messages use JSON over Unix sockets.

Connection	**Inputs and outputs; format and method**	**If something fails**
I1 Setup request Researcher ↔ C1	Dataset/model paths and robot setup JSON → validation/status. JSON configuration through a command-line tool.	C1 lists invalid files/settings and does not start the run.
I2 Validated inputs C1 ↔ C2	Approved demonstration arrays + robot/model settings → saved model/test-report paths and run status. Local function/job call with JSON and file references.	C2 stops on load/memory errors; partial checkpoints are not treated as completed output.
I3 Simulation run C2 ↔ C3	Robot description + checkpoint/settings → validation and test result. During tests: RGB image + joint-state float[n] ↔ velocity float[n]. Local simulator/model calls.	C3 reports failed or incomplete episodes; incomplete tests cannot authorize hardware.
I4 Policy actions C2 ↔ C4	C4 sends run permission/stop; C2 sends model ID, velocity float[n], sequence and observation time. JSON messages over a local socket.	C4 rejects the wrong model, stale data, wrong dimensions or nonfinite values and stops motion.
I5 Test results C3 → C4	Model/robot IDs, episode counts, observed success rates and test status. Versioned JSON report file.	C4 refuses a physical run if the report is missing, incomplete, mismatched or below the approved threshold.
I6 Observations C5 → C2	Timestamped RGB image and measured joint positions/velocities float[n]. Shared memory plus local metadata message.	C5 reports sensor failure; C2 rejects invalid/stale input. No fresh action means the physical run stops.
I7 Checked commands C4 ↔ C5	Limited velocity float[n] + sequence + safety heartbeat → joint positions/velocities float[n], timestamp and acknowledgement bool. JSON messages over a local socket.	C5 refuses expired/replayed commands; C4 stops on stale feedback or failed acknowledgement.
I8 Device messages C5 ↔ robot/camera	Checked velocities + unchanged safety heartbeat → acknowledgement bool, joint-state float[n] and RGB pixels. Wired device API; exact USB/Ethernet protocol follows hardware selection.	C5 stops on lost communication. The controller must time out if C4's heartbeat stops; C5 must not generate it.
I9 Trial start/stop Researcher ↔ C4	Selected model, robot description, approved limits and report-reviewed bool and E-stop-tested bool → allowed/refused/stopped status. Command-line request with JSON settings.	C4 refuses missing checks. Stop requests disable motion; a fault requires a new session check.
I10 Physical emergency stop Researcher → robot	Pressed E-stop switch → latched robot stop. Manufacturer's physical stop circuit, independent of our program.	No physical trial until this stop is tested. Reset follows the lab's procedure.

I7 example for a two-joint test fixture: {"sequence":12,"joint_velocity":[0.02,-0.01],"sent_ms":1000,"heartbeat":61,"valid_for_ms":50}. The vector must match the arm. The timeout is illustrative and needs hardware validation.

5 Data-flow diagram

Figure 2. Offline and physical data flows



C3 exchanges simulated observations and actions with C2 over I3. In a physical trial, C5 returns robot state to C4 over I7. C4 checks the planned motion and stopping distance; if it cannot verify a safe correction, it stops.

Timing and test targets

Target from the previous submission	**How this design handles it**
50 demonstrations; 50 evaluation episodes; 60% success threshold	C1 checks data; C3 compares the new arm with a single-embodiment baseline; C4 checks the report before hardware. These remain draft targets.
0.25 m/s end-effector speed; at least 100 physical control steps	C4 checks motion limits and logs predicted and sent actions. Incomplete logs mean incomplete tests.
Stop within 100 ms if safety heartbeats cease	Use the controller's independent timeout and measure actual stopping time. Hardware stays disabled until the lab confirms this target is achievable.

Proposed 50 ms control-cycle budget: sensing 10 ms, prediction 30 ms, motion checks 5 ms, delivery 5 ms. Reject actions older than 50 ms; send a safety heartbeat every 20 ms. These are untested budgets, subject to model/controller measurements and lab approval. Keep training off the live-inference GPU.

6 Architecture pattern and justification

We use the Week 5 pipeline pattern for adaptation and evaluation, and the embedded sensor-actuator pattern for physical control. The data-preparation steps run in sequence; the robot repeatedly senses, predicts, checks and acts.

Required criterion	**Reason for the choice**
Fit to the problem	The pipeline follows the path from demonstrations to a tested model. The sensor-actuator loop matches repeated robot control.
Team skills	Local modules let us divide model work, evaluation and integration. Joshua's submitted background in backend work, automation and debugging fits setup and safety checks; the proposed ownership still needs team agreement.
Performance and timing	Training stays outside the physical loop. Local communication avoids internet delay, and the selected controller must support the independent stop timeout.
Scalability	The first version uses one active robot and small experiments. We can add datasets and robot descriptions without introducing network services.
Hardware constraints	Use existing team/UC hardware and a pretrained model that fits its memory. Keep robot-specific commands inside C5.

We rejected microservices because separate network services, deployment and distributed failures add work for a two-person project without helping the initial one-robot experiment.

Week 3 constraints: adapt an existing open-source model rather than fund training from scratch; use available team/UC compute; test in simulation first; keep only necessary data and avoid identifiable imagery. Private local/UC storage is our proposed hosting choice. Keyboard commands, readable text output and labels that do not depend on color address accessibility. No exact budget, hosting provider or accessibility standard was set in the earlier submission.

7 Decision log

Choice	**Alternative**	**Reason**
Adapt a pretrained model.	Train a foundation model from scratch.	Fits the project's compute limits and the few-demonstration goal.
Use five local components.	Deploy each part as a network service.	Keeps ownership and interfaces clear without unnecessary deployment work.
Check commands outside the model process.	Send model predictions directly to the robot.	Preserves workspace/speed checks when the model behaves incorrectly.
Keep reviewed data and versioned result files.	Retain all video and overwrite checkpoints.	Supports privacy and lets us reproduce which data and settings produced a result.

References: submitted project background, constraints and user stories; Week 5 architecture lecture; Assignment 5 instructions. Story links follow the five stated user stories, with simulation approval supporting safety stories US-03/04.

World Action Models for Cross Embodiment Robot Control

D1 Detailed Design | Joshua Jerin and Krrish Thakku Suresh
Advisor: Tri Nguyen | CS 5001 |

1 Header, scope, and conventions

Our goal is to adapt a pretrained world action model to a new robot arm with a small set of demonstrations, compare its performance across robots, and test it safely in a controlled lab setting.

Scope. This document details C2 Model, including the experiment registry that holds the project's data model, and C4 Motion safety, whose per-step command check and hardware gate break the project if they are slow or wrong; C1 Experiment setup, C3 Simulation and evaluation, and C5 Robot interface are deferred to D2, although Figure 1 already shows the registry tables they will write.

Conventions. Figure 1 (data model): each box is an entity stored as one table; the header names the entity and, in parentheses, the component that writes it (colour repeats this but carries no extra meaning). Underlined attributes marked PK form the primary key; italic attributes marked FK refer to the entity at the other end of a line. Each line is a relationship, and the numbers at each end give the cardinality at that end as minimum..maximum: 1 means exactly one, 0..1 at most one, 0..* zero or more, 1..* one or more. Figure 2 (algorithm): rounded boxes are processing steps inside C4, dashed boxes are data entering or leaving C4, the octagon stops the trial, the plain box rejects a single proposal, the cylinder is a registry write, solid arrows are control flow labelled with the check result, and the dotted arrow is an asynchronous write. Both figures carry their own legend. Interface IDs I1 to I10 and component IDs C1 to C5 are unchanged from D0, and US-01 to US-05 are the submitted user stories.

Changes since D0. D0 stored every output as an ordinary project file. D1 keeps those files but adds one SQLite registry file that indexes them, so C4 can verify which data, model and report belong together (Section 2). SQLite is a file, not a service, so D0's decision against a separate storage service still holds. The I5 test report remains a JSON file; the registry stores its path and hash. I4, I7 and I9 messages gain an `api_version` field and named error codes (Section 5).

Component and interface reference (from D0)

ID	Component	Responsibility
C1	Experiment setup	Validates the researcher's dataset and robot setup before an experiment starts
C2	Model	Adapts the pretrained model and predicts robot actions
C3	Simulation and evaluation	Tests the model in simulation and compares it with a single-embodiment baseline
C4	Motion safety	Approves physical trials and keeps every command within the approved limits
C5	Robot interface	Passes sensor data in and checked commands out; the only component that talks to the robot

ID	Connects	Carries
I1	Researcher ↔ C1	Setup request and validation result
I2	C1 ↔ C2	Validated demonstrations and settings; saved model and report paths
I3	C2 ↔ C3	Checkpoint for simulation testing; simulated observations and actions

ID	Connects	Carries
I4	C2 ↔ C4	Predicted joint velocities from C2; run permission or stop from C4
I5	C3 → C4	Simulation test report
I6	C5 → C2	Camera image and joint readings
I7	C4 ↔ C5	Checked commands and heartbeat to C5; robot state and acknowledgement back
I8	C5 ↔ robot	Device messages to and from the hardware
I9	Researcher ↔ C4	Trial start or stop request and its result
I10	Researcher → robot	Physical emergency stop, independent of the software

2 Data model, the D1 diagram

The data model belongs to C2's experiment registry, which records provenance for every artifact the project produces. We chose a **relational model in SQLite**. The hardware gate (Section 3.2) must join a checkpoint to its evaluation run, the run's episodes and the robot, and US-01 requires that every checkpoint can be traced to the exact demonstrations, robot description, settings and code that produced it. Foreign keys and unique constraints enforce those links at write time. SQLite runs in-process from Python's standard library with no server, so it adds no deployment work. Large data (demonstration arrays, images, model weights) stays in files on disk; rows hold each file's path and SHA-256 hash so a swapped or edited file is detected. We rejected keeping only per-run JSON files, as D0 implied, because nothing would check that files refer to each other consistently or stop a duplicated episode from inflating a success rate.

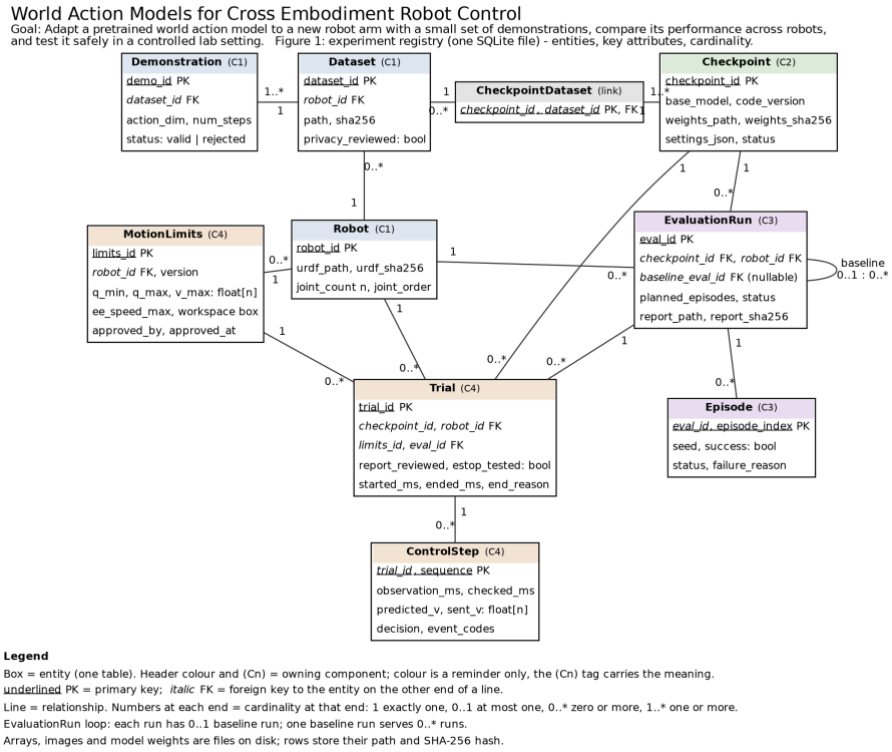


Figure 1. Experiment registry: entities, key attributes and cardinality. Source: Design_Diagrams/fig1_data_model.dot.

Structural decisions

A **checkpoint** is a saved copy of the model at a particular moment: a snapshot of everything it has learned.
A **demonstration** is one recorded example of the robot doing the task correctly, used to teach the model.
An **episode** is one complete attempt at the task, from start to finish, in simulation.
An **evaluation run** is one full simulation test of a model
A **control step** is one tiny moment of a real-robot trial

Decision	Reason	Req.
Demonstration is an entity, not a count on Dataset.	C1 validates each demonstration separately and may reject one for a wrong action dimension or exclude one for privacy. Per-demonstration status records exactly which demonstrations trained a checkpoint.	US-01, US-05, I2
MotionLimits is a separate, versioned entity rather than columns on Robot.	Limits change after advisor review. Each Trial must point to the exact approved version used. Rows are never edited; a change inserts a new version.	US-03, US-04, I9
Checkpoint to Dataset is many-to-many through CheckpointDataset.	A cross-embodiment checkpoint is trained on datasets from several robots, and one dataset is reused by many checkpoints. The checkpoint end is 1..* because an adapted checkpoint must cite at least one dataset. The pretrained base model is stored as the base_model string, not a Checkpoint row, because we did not produce it.	US-01, US-02
Whether a robot was seen in training is derived, not stored.	A robot is seen for a checkpoint when it owns one of that checkpoint's datasets. A stored flag could go stale when datasets are added; a join cannot.	US-02
Episode is an entity; success rate is computed, not stored.	The gate recounts episodes itself, so a hand-edited summary cannot pass. The composite key (eval_id, episode_index) makes a duplicated episode impossible.	US-04, I5
EvaluationRun has an optional self-reference to its baseline (0..1 to 0..*).	Each candidate run points to the single-embodiment baseline run on the same robot and seeds; one baseline run serves many candidates. Baseline runs have no baseline, so the key is nullable.	US-02
Trial references the EvaluationRun that authorized it (1 to 0..*).	Answers the question of which simulation evidence allowed a given physical run.	US-04, I5, I9
ControlStep key is (trial_id, sequence).	One row per control step. The key makes a replayed sequence number impossible to log twice, and the 100-step target is a simple count.	US-03, US-04, I7
Each component writes only its own tables.	Tagged (C1) to (C4) in Figure 1. Other components read but never write them, which keeps ownership as clear as in D0.	D0 Sec. 3

Indexes

Index	Why
Checkpoint.weights_sha256 (unique)	Registering identical weights twice is a duplicate; C2 also looks a checkpoint up by hash when loading weights.
EvaluationRun (checkpoint_id, robot_id, status)	The gate's main lookup: the completed run for this checkpoint on this robot.
Episode (eval_id, episode_index), primary key	The gate counts and sums episodes for one run; the key also blocks duplicates.
MotionLimits (robot_id, version) (unique)	Finds the latest approved limits for a robot; prevents two rows claiming the same version.

Index	Why
ControlStep (trial_id, sequence), primary key	Reads a trial's log in order; rows are append-only.
Dataset (robot_id)	The seen-or-unseen join for US-02.
Not indexed: ControlStep.decision, Episode.success	Scans of one trial or one run touch at most a few thousand rows; an index would only slow inserts.

Size. Realistic: about 1-2 robots, 10 datasets, 500 demonstrations, 30 checkpoints, 60 evaluation runs (3,000 episodes) and 20 trials of up to 2,000 steps each (40,000 ControlStep rows, a few megabytes). At 100 times that, ControlStep reaches about 4 million rows, roughly 1 to 2 GB, which SQLite still handles; the write rate (20 rows per second during a trial) does not change, so scale does not affect the control loop.

3 Core algorithms

Two computations in C4 break the project if they are slow or wrong: the per-step command check, which runs in every 50 ms control cycle, and the hardware gate, which decides whether a physical trial may start at all.

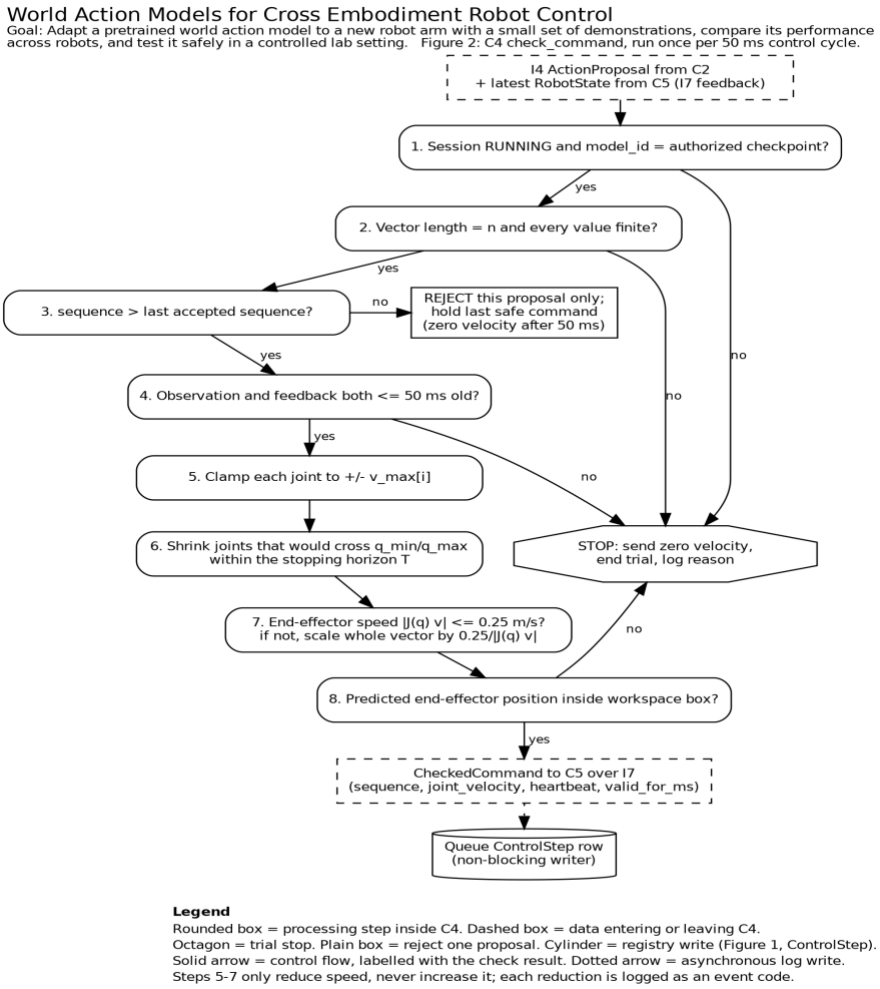
3.1 check_command (C4, every control cycle)

What it is and the problem it solves. check_command turns each action proposed by the model into a command that is guaranteed to stay within the approved limits, or it stops the trial. If it is wrong, the arm can exceed the 0.25 m/s end-effector limit or leave the workspace (US-03, US-04). If it is slow, it eats into the 50 ms cycle (budget 5 ms), commands arrive stale and every trial stops.

Inputs and outputs. All times are integer milliseconds from the same computer's monotonic clock (CLOCK_MONOTONIC); n is the registered joint count; vectors are float64 in the robot's registered joint order.

Name	Type	Source
ActionProposal: model_id, sequence, observation_ms, joint_velocity	str; int64 >= 0; int64; float64[n] rad/s	C2 over I4
RobotState: measured_ms, joint_position, joint_velocity	int64; float64[n] rad; float64[n] rad/s	C5 feedback over I7
Session: checkpoint_id, MotionLimits row, kinematic model, last_sequence, horizon T	loaded once by authorize_trial; last_sequence starts at -1; T in seconds	C4 memory
Output: CheckResult: decision, event_codes, command	enum PASS | LIMITED | REJECT | STOP; list[str]; CheckedCommand or null	to C5 over I7
CheckedCommand: sequence, joint_velocity, sent_ms, heartbeat, valid_for_ms	int64; float64[n] rad/s; int64; int64; int (50)	same fields as D0's I7 example

Figure 2. check_command steps. Source: Design_Diagrams/fig2_check_command.dot.



Procedure. The steps follow Figure 2 and the pseudocode below. Steps 1 to 4 check that the message is valid: a trial is running and the message comes from the approved model (step 1), it has one finite number per joint (step 2), its sequence number is higher than the last one accepted (step 3), and its data is no more than 50 ms old (step 4). If no trial is running or step 3 fails, C4 ignores that one message, and the previous command expires after 50 ms. If any other check fails, C4 stops the trial.

Steps 5 to 7 slow the command down when needed, but never speed it up. Step 5 caps each joint at its speed limit. Step 6 looks ahead by the robot's stopping time T and slows any joint that would otherwise reach the end of its range, minus a safety margin, within that time. A joint already past the margin may only stop or move back, and C4 never makes a joint move that the model did not ask to move. Step 7 uses the arm's geometry to work out how fast the hand would move; if that is over 0.25 m/s, every joint is slowed by the same fraction, so the hand keeps its direction. Step 8 predicts where the hand will be after time T and stops the trial if that point is outside the approved workspace box.

If steps 5 to 7 changed anything, the decision is LIMITED and the reason is recorded. Every step is logged in the background, so writing to disk never delays the robot.

```
function check_command(proposal, state, session, now_ms):
    limits = session.limits          # approved MotionLimits row
    T      = session.horizon_T       # stopping time in seconds
    m      = limits.margin           # safety margin in radians
```

```
    events = empty list

    # Steps 1-4: is the message valid?
    if no trial is running:                          return REJECT("E_NOT_RUNNING")
    if proposal.model_id != session.checkpoint_id:   return STOP("E_WRONG_MODEL")
    if length(proposal.joint_velocity) != n:         return STOP("E_DIM_MISMATCH")
    if any value is NaN or infinite:                 return STOP("E_NONFINITE")
    if proposal.sequence <= session.last_sequence:   return REJECT("E_REPLAY")
    if now_ms - proposal.observation_ms > 50:        return STOP("E_STALE_OBS")
    if now_ms - state.measured_ms > 50:              return STOP("E_STALE_FEEDBACK")

    v = copy of proposal.joint_velocity

    # Step 5: cap each joint's speed
    for each joint i:
        if abs(v[i]) > limits.v_max[i]:
            v[i] = sign(v[i]) * limits.v_max[i]
            add "W_JOINT_LIMITED" to events

    # Step 6: a joint must be able to stop before the end of its range
    for each joint i:
        room_up   = limits.q_max[i] - m - state.q[i]    # radians left going up
        room_down = state.q[i] - (limits.q_min[i] + m)  # radians left going down
        fastest_up   = max(0, room_up / T)       # never below 0: no invented motion
        fastest_down = max(0, room_down / T)
        if v[i] >  fastest_up:   v[i] =  fastest_up;   add "W_RANGE_LIMITED"
        if v[i] < -fastest_down: v[i] = -fastest_down; add "W_RANGE_LIMITED"

    # Step 7: cap hand speed
    hand_speed = length(jacobian(state.q) * v)   # m/s, Pinocchio
    if hand_speed > limits.ee_speed_max:         # 0.25 m/s
        v = v * (limits.ee_speed_max / hand_speed)
        add "W_EE_LIMITED" to events

    # Step 8: hand must stay inside the workspace
    future_hand = hand_position(state.q + v * T) # forward kinematics, Pinocchio
    if future_hand is outside limits.workspace_box: return STOP("E_WORKSPACE")

    session.last_sequence = proposal.sequence
    decision = LIMITED if events is not empty else PASS
    command  = CheckedCommand(proposal.sequence, v, now_ms,
                              session.heartbeat, valid_for_ms = 50)
    queue_log_row(proposal, v, decision, events)  # background, never waits
    return decision, events, command

# STOP sends zero velocity, ends the trial and logs the reason.
# REJECT ignores this message only; the last command expires after 50 ms.
```

Complexity at realistic size. Each step does a fixed amount of work per joint, so the cost grows only with the number of joints n (O(n)), not with how long the trial runs. That is a few hundred arithmetic operations, far below the 5 ms budget. The real risk is Python itself occasionally pausing, so we will time 10,000 test calls and check the slowest 1% before using hardware. At 100 times our data size, trials get longer but each step costs the same, so timing is unaffected; only the log file grows.

Why this algorithm rather than alternatives. We considered four alternatives.

- **Relying only on the robot controller's own joint limits.** Rejected, because the controller does not know about hand speed, our workspace, stale data or which model is running.

- **Rejecting every over-limit command instead of slowing it down.** Simpler, but learned models often go slightly over a limit, so trials would keep stopping before the 100-step target. We slow the command and log it instead.
- **An optimization-based safety filter.** It would find the smallest safe change and could avoid collisions, but it needs a solver running every cycle and collision models we do not have, and it is hard to prove fast enough. We list it as future work.

Edge cases.

Case	Behaviour	Tied to
Empty vector or length ≠ n	STOP, E_DIM_MISMATCH	I4, US-04
NaN or infinite value	STOP, E_NONFINITE	I4
model_id is not the authorized checkpoint	STOP, E_WRONG_MODEL	I4, I9
Duplicate or older sequence number (replay)	REJECT, E_REPLAY; previous command expires after 50 ms, then zero velocity	I7
Gap in sequence numbers	Accept, log W_SEQ_GAP	I4
Observation older than 50 ms / feedback older than 50 ms	STOP, E_STALE_OBS / E_STALE_FEEDBACK	I6, I7, timing budget
Value exactly at a limit; all-zero vector	Passes (comparisons use ≤); zero is always allowed and is the hold command	US-04
Joint already beyond its safety margin	Only zero or inward motion allowed (step 6)	US-04
Arm near a kinematic singularity	No Jacobian inverse is used, so the speed check stays well defined	US-03
First proposal of a trial	last_sequence = −1, so sequence 0 is accepted	I4
Proposal produced on another computer	Not supported: timestamps must share one monotonic clock	I4
Log queue full or write fails	STOP, E_LOG_FAILED (D0: incomplete logs mean incomplete tests)	US-04
C4 itself crashes	Heartbeat stops; the controller's independent timeout stops the robot	I8, D0 Sec. 3

3.2 authorize_trial (C4, before each physical trial)

What it is and the problem it solves. authorize_trial is the single place that enforces simulation evidence before hardware (US-04) and the pre-trial safety checks (US-03). If it is wrong, an untested or under-performing model reaches the real arm.

Inputs and outputs. Input is a TrialRequest over I9: checkpoint_id, robot_id, limits_id, eval_id (str), report_reviewed and estop_tested (bool), operator (str). It reads the registry rows in Figure 1 and the success threshold (float, 0.60, a draft target). Output is a TrialDecision: allowed (bool), trial_id (str or null), success_rate (float 0 to 1), episodes_completed (int) and reasons (list of error codes, empty when allowed).

Procedure. It evaluates every check and returns all failing reasons, not only the first, so the researcher can fix everything in one pass. The checks are: the checkpoint exists with status complete; the evaluation run exists, is complete and matches the requested checkpoint and robot; the run's completed episodes equal planned_episodes and planned_episodes is at least 50; the success rate recomputed from Episode rows meets the threshold; the report file's SHA-256 matches the stored hash, so it was not edited after review; the

limits row belongs to this robot, is approved and is the latest version; both booleans are true; and no other trial is running. If every check passes, it inserts a Trial row and loads the session state used by check_command.

Complexity at realistic size. Counting a run's episodes uses the Episode primary key: O(E) with E = 50, microseconds. At 100 times (E = 5,000) it is still under a millisecond. Hashing the report is O(file size), a few kilobytes. The cost that does matter is the model weights, which can be several gigabytes, so hashing them takes tens of seconds; the gate therefore compares file size and modification time against the registry, and C2 verifies the full hash when it loads the weights.

Why this algorithm rather than alternatives. A manual checklist alone was rejected because nothing enforces it. Trusting a success rate stored in the report was rejected because a hand edit or duplicated episode could pass. The open question is the statistical test. Comparing the point estimate with 60%, as the D0 target implies, passes 30 successes out of 50, yet the 95% Wilson interval for 30 of 50 is about 0.46 to 0.72, and a model whose true success rate is 50% would still pass about 10% of the time. Requiring the Wilson lower bound to clear the threshold is stricter but needs either more episodes or a lower threshold

Edge case	Behaviour
Zero completed episodes	Refuse with E_REPORT_INCOMPLETE; no division is attempted
Success rate exactly at the threshold	Passes (≥) x
Failed or incomplete episodes	Not counted as completed, so the run is incomplete and refused (D0 I3)
Run evaluated a different checkpoint or robot	Refuse with E_REPORT_MISMATCH
Report file changed after its hash was stored	Refuse with E_REPORT_TAMPERED
Newer limits version approved since the request was written	Refuse with E_LIMITS_NOT_CURRENT; resubmit with the new limits_id
A trial is already running	Refuse with E_TRIAL_ACTIVE (one active robot, D0 Sec. 6)
Previous trial ended in a fault	A fresh authorize_trial is required (D0 I9)

4 Build-versus-reuse decisions

Licenses below are for the projects as we understand them;

Piece	Build or reuse	Library or service	License	Reason
Pretrained world action model (C2)	Reuse	candidates: Octo; openpi (π0); OpenVLA	Octo MIT; openpi Apache-2.0; OpenVLA code MIT, weights Llama 2 Community License	Training from scratch is outside our compute (D0 decision 1). Choose by GPU memory, whether fine-tuning code is published, and action space.
Fine-tuning and inference code (C2)	Reuse, then adapt	The chosen model's own repository	Same as model	Its data loaders and action heads are tested; we change only configuration and the robot-specific head.
Deep-learning framework (C2)	Reuse	JAX (Octo, openpi) or PyTorch (OpenVLA)	Apache-2.0 / BSD-3-Clause	Follows the model; mixing frameworks adds work for no gain.

Piece	Build or reuse	Library or service	License	Reason
Experiment registry (C2)	Reuse engine, build schema	SQLite via Python sqlite3	Public domain / PSF	Mature, in standard library, no server (Section 2).
Dataset format (C1, C2)	Reuse	The model's format (RLDS or LeRobot)	Apache-2.0	Converting once at import avoids a custom loader.
Forward kinematics and Jacobian (C4)	Reuse	Pinocchio	BSD-2-Clause	Reads URDF directly; fast C++ core with Python bindings; widely used. Hand-written kinematics is an easy place to make silent errors.
check_command logic (C4)	Build	NumPy for vector maths	BSD-3-Clause	Project-specific and safety-critical; must be small enough to read and test line by line.
authorize_trial gate (C4)	Build	sqlite3	PSF	The rules are specific to our requirements.
Wilson interval (C4)	Reuse	statsmodels proportion_confint	BSD-3-Clause	Standard, tested implementation; only used if the lower-bound rule is chosen.
Message schemas and validation (C2, C4)	Reuse	pydantic v2	MIT	Typed models generate the error for a missing or mistyped field; compiled core is fast enough for 20 messages per second
Local transport (I4, I7, I9)	Reuse	Python socket (Unix domain sockets) and json	PSF	Already chosen in D0; no extra dependency.
Heartbeat timer and log writer (C4)	Build on standard library	threading, queue	PSF	A few dozen lines; behaviour must match D0's 20 ms heartbeat exactly.
Command-line tools (I1, I9)	Reuse	argparse	PSF	Keyboard-only text interface meets D0's accessibility note.
Tests, including property tests of the checker	Reuse	pytest, Hypothesis	MIT / MPL-2.0	Hypothesis generates random proposals to check that output never exceeds a limit.

5 API contract

All C4 methods are JSON messages over C4's Unix domain socket; register_checkpoint is a Python method of C2's registry module. Every message is a JSON object with `api_version` (str, "MAJOR.MINOR") and `type` (str). A failed request returns `"ok": false` and an `error` object with `code` (str), `message` (str, human readable) and `action` (one of refuse, reject, stop). Callers must treat any unknown error code as stop.

5.1 authorize_trial (I9, researcher CLI to C4)

Parameter	Type	Required	Valid range
checkpoint_id, robot_id, limits_id, eval_id	str	Yes	Must exist in the registry

Parameter	Type	Required	Valid range
report_reviewed, estop_tested	bool	Yes	Must be true to be allowed
operator	str	Yes	1 to 64 characters
note	str	No	Up to 500 characters; stored on the Trial row

Output: `{ok: bool, allowed: bool, trial_id: str | null, success_rate: float 0–1, episodes_completed: int, reasons: list[str]}`.

5.2 check_command (I4 in from C2, I7 out to C5)

Parameter	Type	Required	Valid range
model_id	str	Yes	Equals the authorized checkpoint_id
sequence	int64	Yes	Greater than the last accepted value
observation_ms	int64 (ms, monotonic)	Yes	now − observation_ms ≤ 50
joint_velocity	float64[n] (rad/s)	Yes	Length n, all finite; larger values are limited, not refused

Output: `{ok: bool, decision: "PASS"|"LIMITED"|"REJECT"|"STOP", event_codes: list[str], command: {sequence: int64, joint_velocity: float64[n] rad/s, sent_ms: int64 ms, heartbeat: int64, valid_for_ms: int ms} | null}`. command is null for REJECT and STOP.

5.3 stop_trial (I9)

Inputs: trial_id (str, required), reason (str, optional, up to 200 characters). Output: `{ok: true, status: "stopped", ended_ms: int64}`. Stopping always sends zero velocity first and is idempotent: stopping an already-stopped trial returns E_NO_TRIAL with action none.

5.4 register_checkpoint (C2 registry, used when C2 finishes adaptation over I2)

Inputs: weights_path (str, required, existing file), dataset_ids (list[str], required, at least one, all registered), base_model (str, required), code_version (str, required, 40-character git commit hash with no uncommitted changes), settings (object, required, saved as settings_json). Output: `{ok: true, checkpoint_id: str, weights_sha256: str (64 hex), status: "complete"}`. A checkpoint whose training did not finish is never registered as complete (D0 I2).

Error responses

Code	Method	When it happens	Caller action
E_API_VERSION	any	Major version differs from C4's	Refuse
E_NOT_RUNNING	check_command	No authorized trial is active	Reject
E_WRONG_MODEL	check_command	model_id ≠ authorized checkpoint	Stop
E_DIM_MISMATCH / E_NONFINITE	check_command	Length ≠ n, or NaN/Inf value	Stop
E_REPLAY	check_command	sequence ≤ last accepted	Reject
E_STALE_OBS / E_STALE_FEEDBACK	check_command	Data older than 50 ms	Stop
E_WORKSPACE	check_command	Predicted position outside box	Stop
E_LOG_FAILED	check_command	Log queue full or write error	Stop

Code	Method	When it happens	Caller action
W_JOINT_LIMITED / W_RANGE_LIMITED / W_EE_LIMITED / W_SEQ_GAP	check_command	Steps 5, 6, 7 reduced speed; or sequence gap (warnings, decision LIMITED or PASS)	Continue
E_CHECKPOINT_PARTIAL, E_REPORT_MISSING, E_REPORT_INCOMPLETE, E_REPORT_MISMATCH, E_REPORT_TAMPERED, E_BELOW_THRESHOLD	authorize_trial	The corresponding check in Section 3.2 fails	Refuse
E_LIMITS_NOT_APPROVED / E_LIMITS_NOT_CURRENT, E_REVIEW_MISSING, E_ESTOP_UNTESTED, E_TRIAL_ACTIVE	authorize_trial	Limits, booleans or active-trial check fails	Refuse
E_NO_TRIAL	stop_trial	trial_id unknown or already ended	None (stop is idempotent)
E_FILE_MISSING, E_DATASET_UNKNOWN, E_DUPLICATE_WEIGHTS, E_DIRTY_CODE	register_checkpoint	Weights file absent; dataset not in registry; same hash already registered; code_version has uncommitted changes	Refuse

Example request and response (check_command, two-joint test fixture from D0)

```
// C2 -> C4 over I4
{"api_version": "1.0", "type": "action_proposal",
 "model_id": "ckpt_0007", "sequence": 12, "observation_ms": 987,
 "joint_velocity": [0.02, -0.01]}

// C4 -> C5 over I7 (proposal within limits)
{"api_version": "1.0", "type": "checked_command", "ok": true,
 "decision": "PASS", "event_codes": [],
 "command": {"sequence": 12, "joint_velocity": [0.02, -0.01],
             "sent_ms": 1000, "heartbeat": 61, "valid_for_ms": 50}}

// Same proposal arriving 63 ms after its observation
{"api_version": "1.0", "type": "checked_command", "ok": false,
 "decision": "STOP", "event_codes": ["E_STALE_OBS"], "command": null,
 "error": {"code": "E_STALE_OBS", "action": "stop",
           "message": "observation is 63 ms old (limit 50 ms)"}}
```

Versioning. Every message carries `api_version` as MAJOR.MINOR and C4 refuses a request whose MAJOR differs from its own, while the registry records its schema version in SQLite's `PRAGMA user_version` and changes only through numbered migration scripts; a breaking change, which needs a new MAJOR, is removing or renaming a field, changing a field's type, unit or joint order, or making C4 accept an input it used to refuse, whereas adding an optional field or a new error code is a MINOR change.

6 Technology choices with justification

Backend language: Python 3.11 or later. *Team skill:* we have used many Python machine-learning libraries and frameworks in personal projects and hackathons. *Licensing:* Python's license is permissive, and every library in Section 4 works with Python. *Community:* Python is the main language of robot-learning research,

so the models, Pinocchio, MuJoCo and robot drivers all come with Python examples. *Performance:* the heavy maths runs in fast compiled libraries; Python's own overhead is the only risk, and we will measure it against the 5 ms budget. *Cost and hosting:* free, and already installed on lab and UC machines. *Rejected:* C++ for C4. It has more predictable timing, but two languages is too much for a two-person team. We will switch only if Python is measured too slow.

Machine-learning framework: PyTorch. *Team skill:* The team is adequately skilled with these tools. *Licensing:* both are free, open-source licenses (JAX: Apache-2.0, PyTorch: BSD-3-Clause). *Community:* both are widely used; we pick the one the chosen model's official code uses. *Performance:* both are fast on a GPU; prediction must fit the 30 ms budget, which we will measure. *Cost and hosting:* free, and both run on UC GPUs. *Rejected:* the other framework, because converting a pretrained model between frameworks is a project in itself.

Simulator: MuJoCo. *Licensing:* free and open source (Apache-2.0). *Community:* maintained by Google DeepMind, widely used, with ready-made robot models. *Performance:* runs fast on a normal CPU, so many tests can run at once without using the model's GPU. *Cost and hosting:* free on any lab or UC machine. *Rejected:* NVIDIA Isaac Sim, which has more realistic images but needs a high-end GPU and has a proprietary license; and PyBullet, which is no longer actively developed. [TEAM: if realistic camera images matter a lot for your model, check this choice with your advisor.]

Database: SQLite plus files. *Team skill:* Taken college courses and built applications; we use plain SQL, so there is little to learn. *Licensing:* public domain. *Community:* one of the most widely used databases, well documented, and built into Python. *Performance:* our busiest table gets only 20 new rows per second, which is easy for SQLite, and logging happens in the background. *Cost and hosting:* just a file in our private project storage, with no server to run. *Rejected:* PostgreSQL. It is built for many users writing at once, but we have one machine and one trial, and running a database server goes against D0's "no separate storage service" decision.

Messaging and hardware interface: JSON over local sockets; the robot's own interface inside C5. *Licensing:* sockets and JSON are built into Python; the robot interface is manufacturer SDK. *Community:* sockets and JSON are universal; ROS 2 has the largest robotics community. *Performance:* local messages take well under 1 ms, inside the 5 ms budget. *Cost and hosting:* free and local, with no network needed. *Rejected:* ROS 2 for messages between our components. It has good tools and robot drivers, but it is a lot to learn on top of the model and simulator. If our robot's driver needs ROS 2, C5 will handle it, and the rest of the system stays the same.

Compute and hosting: training on UC research computing cluster or lab GPU, the model, C4 and C5 run on one lab computer connected to the robot. *Licensing:* university resources under UC's usage rules. *Community:* support from UC's research computing staff and documentation. *Performance:* running on one computer means C2 and C4 share the same clock, and training never slows down the live robot's GPU. *Cost and hosting:* no cloud bills, and camera data stays on university storage, which protects privacy (US-05). *Rejected:* rented cloud GPUs, because they cost money we haven't budgeted and would send camera data off campus. There is no graphical interface; command-line tools keep D0's keyboard-and-text accessibility approach.

World Action Models for Cross Embodiment Robot Control

D2 Detailed Design | Joshua Jerin and Krrish Thakku Suresh
Advisor: Tri Nguyen | CS 5001 | October 14, 2026

1 Header, scope, and D2 overview

Our goal is to adapt a pretrained world action model to a new robot arm with a small set of demonstrations, compare performance across robots, and test safely in a controlled lab.

Overview. C1 validates data and robot setup; C2 adapts the policy; C3 compares it with a single-embodiment baseline; C4 authorizes and checks physical motion; C5 alone talks to the robot. D1 detailed the registry in C2 and safety algorithms in C4. D2 finishes C1, C3 and C5, then checks all five components for failures and requirement coverage.

Review status. New details, edge analysis, traceability and rationale are proposals for team validation, not implementation results or approved team decisions. Actual peer-review evidence is still missing.

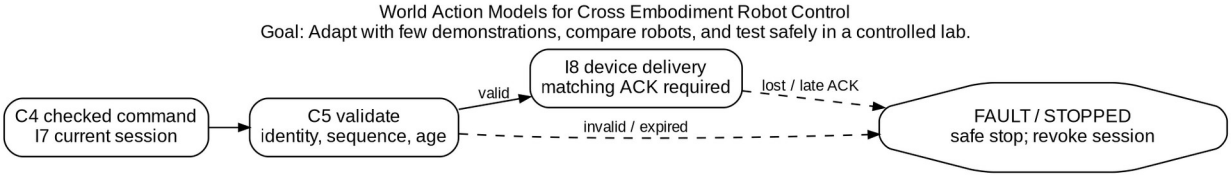


Figure 1. C5 error path. Rounded boxes are input/processing steps; the octagon is a stopped state. Solid arrows are normal flow; dashed arrows are faults. Recovery requires operator reset and fresh authorization. I10 is an independent physical E-stop.

Conventions. C = component; I1-I10 retain D0 meanings; PK/FK = primary/foreign key. Joint vectors are finite float[n] in registered order: position rad, velocity rad/s. Times are same-host monotonic milliseconds. IDs/paths/versions are strings; counts/sequences are nonnegative integers; flags are Boolean. Proposed D0 owners: Joshua C1/C4; Krrish C2/C3/C5.

Known gaps. G1: choose model, robot, driver and action adapter. G2: confirm task, seeds, baseline and pretraining exposure. G3: validate full-path/stopping safety, watchdogs and real braking time. G4: approve privacy/access rules and persistence recovery. G5: supply real peer feedback. All numerical targets remain draft; no hardware trial is authorized here.

2 Finished components

2.1 C1 Experiment setup

Data. C1 owns Robot(robot_id PK, URDF path/hash, n, joint_order), Dataset(dataset_id PK, robot_id FK, path/hash, privacy_reviewed), and Demonstration(demo_id PK, dataset_id FK, action_dim, num_steps, status). One robot has many datasets; one dataset has many demonstrations. A proposed hashed manifest adds per-file hash, rejection reason and privacy reviewer/method/time. Registered inputs are immutable.

Behavior. Authenticate the caller and restrict paths to the project root. Validate robot loading through C2/I3; scan every demonstration for dimensions, units, finite values and timestamps. Any dimension mismatch blocks the whole training request. Keep images restricted until exclusion, anonymization or approved access restriction is reviewed. Freeze/recheck hashes, publish files atomically, then commit rows; only ready data reaches C2. Corrections create a new version. The draft target is at least 50 valid demonstrations.

Interface	Inputs and result	Errors and recovery
I1 validate_setup	api_version/type, request_id and robot/dataset/config paths (str). Returns ready/invalid, IDs, counts and errors[{file, code, expected, actual, recovery}].	E_PATH, E_SCHEMA, E_DIM_MISMATCH, E_PRIVACY_PENDING, E_TOO_FEW_DEMOS: refuse; list every issue. Correct and revalidate; zero training steps.
I2 start_adaptation	request_id, dataset_ids, robot_id, base_model_ref, settings_ref/hash. Returns job_id/status, then checkpoint_id/report refs. Same ID/payload returns existing job.	E_DATASET_NOT_READY, E_HASH_MISMATCH, E_REQUEST_CONFLICT: refuse. Storage/DB busy: bounded retry; never publish ready or complete partial output.

2.2 C3 Simulation and evaluation

Data. C3 owns EvaluationRun(eval_id PK, checkpoint_id/robot_id FKs, baseline_eval_id nullable FK, planned_episodes, status, report_path/hash) and Episode((eval_id,index) PK, seed, success, status, failure_reason). A frozen evaluation spec records task/version, seeds, horizon, scoring rule, adapter/config/code hashes and role=candidate|baseline. Only candidates require baseline_eval_id.

Behavior. Verify checkpoint and robot identity; match candidate/baseline task, robot, seeds, horizon, scoring and comparable data/compute budgets. Keep evaluation data held out. Score ordinary task failures as completed success=false; infrastructure failures remain incomplete. Retry only predefined infrastructure failures with the same seed, preserving attempts. A single coordinator prevents duplicate episodes. Publish a hashed report only after every planned episode and file/DB commit succeeds. C4 recounts episodes; the draft gate is at least 50 episodes and >=60% success.

Interface	Inputs and result	Errors and recovery
I3 validate_robot / evaluate	Robot/spec hash or checkpoint_id, robot_id, spec_ref, role and baseline_eval_id. Returns dimensions/frame IDs or eval_id, status/counts and report ref/hash. infer accepts RGB uint8[H,W,3] and state[n], returns velocity[n].	E_SIM_LOAD, E_ADAPTER_MISMATCH, E_BASELINE_MISMATCH, E_ACTION_INVALID: refuse/fail. E_SIM_CRASH/E_TIMEOUT: bounded recorded retry; exhaustion stays incomplete.
I5 report	Complete versioned JSON: model/robot/run IDs, task/seeds, counts/successes/rates, baseline, exposure labels and hashes. C4 resolves by eval_id.	Missing, incomplete, edited or mismatched report: refuse E_REPORT_*. No report carries hardware permission; only C4 authorizes it.

Scientific limit. The D1 dataset join proves only exposure during project adaptation. Record base-model exposure as known seen, documented unseen or unknown. An adapted target is not unseen during adaptation; unknown pretraining exposure cannot support a never-seen claim.

C5 Robot interface and shared controls

2.3 C5 Robot interface

Data and behavior. C5 has no new registry tables. Volatile RobotSession stores session/trial/robot IDs, adapter/config hash, state, sequence/heartbeat and sensor/ACK times. Bounded ObservationFrame buffers hold RGB and measured joints; retain no images without C1 review. States: DISCONNECTED -> READY -> ARMED -> ACTIVE; any fault enters STOPPED/FAULT. Only an operator reset, repeated preflight and fresh C4 authorization rearm it.

Preflight verifies actual hardware, units, joint order, sensor alignment and independent stop/expiry capability. Accept only authenticated, current-session, finite, fresh C4 commands. Do not queue old motion, alter checked commands, renew timestamps or invent heartbeats. On error request the documented safe stop, revoke permission and notify C4. Zero velocity alone does not prove a physical stop.

Interface	Contract and failure handling
I6 observe	{version, session_id, frame_id, observation_ms, shared-memory image_ref/shape/dtype, q[n], velocity[n]}. Consumers lease/copy before reuse. Invalid/stale sensors stop the trial; dropped old frames cannot be relabelled fresh.
I7 deliver	CheckedCommand envelope binds session/trial/robot/config plus sequence, velocity[n], sent_ms, heartbeat, valid_for_ms=50. Require 0 <= age < lifetime and increasing sequence. ACK identifies session/sequence and actual driver acceptance. E_AUTH/E_SESSION/E_REPLAY/E_EXPIRED/E_CLOCK or missing ACK: refuse/fault; never refresh expiry.
I7 heartbeat / feedback	C4 sends {session_id, trial_id, counter, issued_ms} every 20 ms independently of commands, only while session health is valid. C5 forwards unchanged; stale/repeated heartbeats do not refresh watchdogs. A heartbeat never renews command lifetime. Feedback carries measured state/time and acknowledged sequence.
I8 adapter; I10 E-stop	connect/read_state/read_camera/send_checked/send_heartbeat/request_safe_stop/disconnect. Lost device, negative/late ACK or unsupported watchdog enters FAULT. Exact SDK is G1. I10 remains the manufacturer's independent latched physical stop, tested this session.

2.4 Authentication, notifications, configuration and logging

C	Access and configuration	Notifications and logs
C1	Authorized OS caller; project-root paths; versioned import/privacy policy.	CLI lists all validation failures. Persist manifest, counts and review evidence; never log raw images.
C2	Only verified job/inference peers; hash-bound datasets/model/settings; training off live GPU.	Job progress/failure and permission loss. Persist checkpoint provenance; partial weights never complete.
C3	Requests through C2; validated artifacts; frozen task/seeds/horizon/scoring.	Completed/planned and incomplete reasons. Persist every outcome/attempt and report hash.
C4	Verified I9 operator and I4/I7 peers; one active trial; immutable approved limits.	Allowed/refused/stopped with codes. Log proposed/sent actions and events; queue/write failure stops.
C5	Only verified C4/session owns command path; versioned device/unit/frame mapping.	Sequence-bound ACK/state/fault events; bounded metadata-only logs. Audit failure disables trials.

Use restricted local sockets and verified OS peer identity; a JSON operator name is not authentication. A compromised same-account process is outside the supported threat model unless service identities are isolated. Versioned IDs/manifests replace duplicated I2/I3 parameters; C5 sees no training internals. Session binding requires a coordinated new API major version. Responses use {ok, status, error{code,message,action,recoverable}}; action is refuse/reject/stop/none. Unknown live errors stop. Stop is idempotent and separate from an ordinary checked command.

3 Edge cases and error handling

Proposed for team validation. B = boundaries; T = timing/concurrency; I = bad/missing input; X = external dependencies. Every component covers all four categories. Recovery never implies automatic physical resumption. Figure 1 shows the hardware stop path.

ID / category	Edge case	Named handler, outcome and recoverability
EC01 C1 B	0/49 demos; empty sequence.	validate_setup refuses E_TOO_FEW_DEMOS/E_EMPTY; zero training. Recover after corrected data meets the draft target.
EC02 C1 T	Files change; duplicate start.	commit_dataset rehashes; start_adaptation deduplicates identical request IDs. Conflicts fail. Recover with a newly validated request.
EC03 C1 I	Wrong dimension/units; NaN; privacy pending.	validate_setup refuses the whole set and lists errors. privacy_review excludes/reviews images. Recover by correction, not silent filtering.
EC04 C1 X	Simulator, disk or DB unavailable.	commit_dataset/validate_robot refuse; bounded retry for transient busy. No ready data. Recover after dependency repair and revalidation.
EC05 C2 B	GPU memory exhausted; empty weights.	train_job fails E_OOM/E_CHECKPOINT_PARTIAL; no complete registration. Recover with an explicitly revised configuration/checkpoint.
EC06 C2 T	Training competes; stale inference.	run_scheduler refuses live-GPU contention; check_command stops stale actions. Recover offline or through fresh trial authorization.
EC07 C2 I	Wrong model/shape; bad artifact hash.	load_checkpoint refuses bad provenance; check_command stops wrong model/shape. Recover using validated artifacts; no automatic substitution.
EC08 C2 X	GPU crash; checkpoint write fails.	train_job marks failed; live supervisor revokes session. Recover after restart/revalidation; partial output stays incomplete.
EC09 C3 B	0 or 49/50 scored; 29 vs 30 successes.	finalize_evaluation avoids zero division; incomplete stays blocked. authorize_trial applies draft 30/50 threshold only after all checks.
EC10 C3 T	Duplicate/late result; selective retry.	episode_coordinator rejects duplicates/late results. Retry only predefined infrastructure faults with the same seed; never rescore task failures selectively.
EC11 C3 I	Unmatched baseline; unknown exposure.	validate_eval_spec refuses mismatches. Unknown exposure stays labelled unknown. Recover with matched specification or documented evidence.
EC12 C3 X	Simulator crash; report write fails.	evaluate/finalize_evaluation retries eligible faults within policy; otherwise incomplete. Recover dependencies and record attempts before a new complete report.
EC13 C4 B	Boundary/zero action; unsafe intermediate path.	check_command limits or stops. Endpoint-only checking is insufficient; hardware remains disabled pending validated swept/stopping-envelope checks.
EC14 C4 T	Replay/future time; stale state; competing start.	check_command rejects replay without refreshing expiry, stops bad time/stale data. authorize_trial serializes starts. Recover through a new session.
EC15 C4 I	NaN/shape; missing review or limits.	check_command stops invalid vectors; authorize_trial refuses missing approvals. Correct inputs and repeat checks; unknown live errors stop.
EC16 C4 X	Log overflow/write failure; C4 crash.	log_health triggers E_LOG_FAILED stop; independent controller timeout covers crash. Mark log incomplete; repair and retest before authorization.
EC17 C5 B	At expiry; full buffer; sequence rollover.	validate_command rejects age >= lifetime; buffer_manager drops old frame. Rollover requires new session. Recover only with fresh data/session.
EC18 C5 T	Old ACK/session; heartbeat but no commands.	command_watchdog independently expires velocity; feedback_watchdog faults on late ACK/state. Heartbeats cannot renew commands. Operator reset/preflight required.
EC19 C5 I	API/unit/order/image/state mismatch.	adapter_preflight refuses incompatible schema/mapping; observe faults invalid sensors. Correct configuration and repeat checks; never infer units.
EC20 C5 X	Disconnect, driver fault, absent watchdog.	enter_fault requests safe stop and revokes permission. Verified independent stop covers link loss. Repair, reconnect and retest before rearming.

4 Requirements traceability

Proposed for team validation. All rows have must-fix priority (M), except NF12 (worth considering). Text is condensed from submitted User Stories, D0 and D1; IDs are unchanged. “Covered” means a design mechanism exists, not a passing test. Fig. 1 shows the hardware error path.

4.1 Functional requirements

ID (M)	Requirement	Specific element	How and remaining gap
US-01	Adapt a pretrained policy to a new arm with few demonstrations.	C1 §2.1; C2 registry; EC01/03/05/07.	Validated data and linked checkpoint. G1: model/arm/action adapter unconfirmed.
US-02	Compare trained-on/unseen robots with a single-embodiment baseline.	C3 §2.2 exposure and matched spec; EC10/11.	Matched scores and separate exposure labels. G2: unknown pretraining exposure blocks “never seen.”
US-03	Keep motion inside fixed workspace/speed limits to protect people.	C4 check_command; C5 §2.3; Fig. 1; EC13/14/18.	Independent filtering and expiry. G3: full path, braking and exclusion-zone validation missing.
US-04	Independently reject/clamp unsafe commands before the controller.	C4 authorize_trial; C5 deliver/adapter; EC15/16/20.	Only current checked commands reach hardware. G3: controller/fault-injection evidence missing.
US-05	Exclude, anonymize or restrict identifiable imagery before retention.	C1 privacy_review; C5 volatile buffers; EC03/19.	Review gate and image-free logs. G4: approve procedure and inspect all retained copies.
AC-01.1	50 demos produce a linked checkpoint and 50-episode baseline report.	C1 manifest; CheckpointDataset; C3 Episode/report; EC01/07/09.	Hashes tie data/config/robot/code to results. End-to-end selected-model test still needed.
AC-01.2	At >=60% over 50 episodes, permit hardware-candidate marking.	C3 complete report; C4 recount/gate; EC09/15.	Threshold plus review/preflight gates. Clarify evaluated status versus hardware eligibility.
AC-01.3	Below 60%, refuse marking; source calls status “not evaluated.”	C4 E_BELOW_THRESHOLD; C3 retained result; EC09/10.	Refuse eligibility without deleting results. Proposed status “evaluated_below_threshold” needs approval.
AC-01.4	Any dimension mismatch: zero training, no checkpoint, list every file.	C1 whole-set validate_setup/I1; EC03.	All-error scan blocks I2 before a job starts. Verify a mixed valid/invalid dataset.
AC-02.1	100 steps: all commanded positions in bounds; speed <=0.25 m/s.	C4 limits/ControlStep; C5 measured state; EC13/16.	Add derived pose/speed and frame/model versions to logs. Actual trajectory/measurement proof missing.
AC-02.2	Unsafe workspace prediction produces safe output and a logged event.	C4 limit/stop event; C5 safe stop; Fig. 1; EC13.	Never forward unsafe motion. Source says clamp; D1 stops: approve conservative semantics and test path.
AC-02.3	Lost heartbeat stops motion within 100 ms and blocks further actions.	C4 timer; C5 relay/expiry; controller; EC14/18/20.	Independent timeout and revoked session. Measure reaction plus braking; restart alone is insufficient.

Nonfunctional constraints and reverse check

NF IDs organize inherited D0/D1 constraints; they are not newly claimed stakeholder requirements. The numerical limits are draft targets. No safety or performance result is claimed.

ID / priority	Constraint	Element, how and gap
NF01 M	Trace checkpoints to exact data, robot, settings and clean code.	C1 manifest/C2 registry; EC02/07. Hashes bind provenance; loaded-model identity attestation remains to implement.
NF02 M	One active robot/trial; C4 independent; C5 alone drives hardware.	C4 atomic gate/C5 session ownership; Fig. 1; EC14/18. Test competing starts and direct-driver access protection.
NF03 M	50 ms cycle: sense 10, infer 30, check 5, deliver 5 ms.	C2 budget/C4 freshness/C5 expiry; EC06/14/17. Measure tails; observation age <=50 ms, command age <50 ms.
NF04 M	20 ms safety heartbeat and independent controller timeout.	C4 timer/C5 relay and separate command watchdog; EC18/20. Verify protocol and actual stop-time margin.
NF05 M	Log every proposal, sent action and limit event; incomplete logs invalidate tests.	C4 queue/log_health/ControlStep; EC16. Stop on overflow/write failure; add fault records and test durable completeness.
NF06 M	Local JSON sockets, same-host clock, explicit units/order and versioning.	§2.3-2.4 typed envelopes/session binding; EC14/19. Coordinate new major version; remote clocks unsupported.
NF07 M	SQLite plus files; foreign/unique keys; component-owned writes.	C1/C3 registry APIs and publication; EC04/10/12. Short transactions prevent duplicates; test file/DB crash recovery.
NF08 M	Reuse pretrained open-source model and available team/UC compute.	C2 base_model_ref/run_scheduler; EC05/06/08. No training on live GPU; verify license, capacity and model fit.
NF09 M	Collect only needed data, use controlled/private storage, avoid identifiable recording.	C1 privacy manifest/C5 buffers and redaction; EC03/19. Define retention/access and inspect caches/backups.
NF10 M	Keyboard CLI, readable text, labels independent of color.	I1/I9 status and errors; Fig. 1; EC03/15. Text labels/codes; keyboard/usability check remains.
NF11 M	Simulation first; approved limits, reviewed report and current-session E-stop test.	C4 authorize_trial/MotionLimits/C5 preflight; EC15/20. Fail closed; recorded lab and hardware evidence required.
NF12 W	Small local deployment; D1 discusses 100x data/log growth.	Indexed registry/bounded queues; EC04/12/16. Scale is an estimate, not measured capacity; benchmark disk contention.

4.2 Reverse component check

C	Requirements supported	Specific check
C1	US-01/02/05; AC-01.1/01.4; NF01/07/09/10.	§2.1; EC01-04. Validated provenance and privacy; no model-quality claim.
C2	US-01/02; AC-01.1; NF01/02/03/06/08.	D1 registry and §2.1; EC05-08. Adaptation, artifacts and live model identity.
C3	US-01/02/04; AC-01.1-01.3; NF07/08/11/12.	§2.2; EC09-12. Supplies complete comparative evidence, not hardware permission.
C4	US-03/04; AC-01.2/01.3 and 02.1-02.3; NF02-07/11.	D1 algorithms + Fig. 1; EC13-16. G3 remains a physical-safety blocker.
C5	US-03/04/05; AC-02.1-02.3; NF02-06/09/11.	§2.3; EC17-20. Preserves command/heartbeat limits at the actual device boundary.

5 Design rationale

Proposed for team validation. These nine entries are prompts for the team's own rationale. D0 Section 7 already records pretrained adaptation, local components, independent safety and versioned private artifacts. D1 remains unchanged. No decision is attributed to peer feedback without evidence.

DR01 Inherited from D1. SQLite plus hashed files over independent JSON-only records. Foreign keys and unique episode keys enforce provenance; the trade-off is schema/migration work. D2 retains this choice and adds crash-safe publication. NF01/07; EC02/10/12.

DR02 Proposed. Whole-set validation over silently dropping incompatible demonstrations. AC-01.4 requires zero training for any dimension mismatch. The trade-off is correction work before training; it preserves the acceptance criterion and a reproducible dataset. US-01/05; EC03.

DR03 Proposed. Frozen matched evaluation specifications over ad hoc test settings. Common tasks, seeds, horizons and scoring make comparisons interpretable. The trade-off is a new version when conditions change; outcomes cannot be selectively retried to improve the score. US-02; EC10/11.

DR04 Proposed correction. Separate adaptation/pretraining exposure labels over a single seen/unseen flag. A dataset join cannot reveal base-model training history. The trade-off is reporting "unknown" and limiting the generalization claim until evidence exists. US-02; G2; EC11.

DR05 Proposed. A thin typed C5 adapter and immutable references over shared SDK internals and repeated parameter lists. This preserves D0's hardware boundary and reduces unit/order drift. The trade-off is an explicit adapter/configuration validation step. NF02/06; EC19/20.

DR06 Proposed correction. Independent command expiry plus unchanged heartbeats over heartbeat-only liveness. C4 can remain alive while valid actions stop. The trade-off is more false stops under delay; stale motion is worse. Verify both paths and actual braking. AC-02.3; EC18.

DR07 Proposed safety correction. Keep hardware disabled pending a validated full-path/stopping envelope over trusting D1's safe endpoint. The intervening path or braking trajectory may leave the workspace. The trade-off is extra validation or delayed hardware trials. This is an open gap, not an implemented guarantee. US-03; EC13.

DR08 Proposed. Bounded nonblocking logs with explicit failure handling over synchronous writes or silently dropped audit records. The trade-off is stopping when the writer/queue fails; otherwise timing or log completeness is lost. Reserve fault-report capacity and stress-test it. NF05; EC16.

DR09 Proposed. Local OS access checks and session binding over unauthenticated sockets or a new web identity stack. The trade-off is trust in the lab host/account and verified process isolation. A compromised operator account is unsupported unless service identities are separated. NF02/06/09; EC19.

Corrections requiring explicit team acceptance

Keep US-03 = nearby people and US-04 = hardware despite swapped source cross-references. Separate "evaluated" from "hardware eligible." Resolve workspace clamp-versus-stop, STOP's null command versus the separate safe-stop event, and action="none" in the error enum. Add queue-health checks and early fault/reject logging absent from D1 pseudocode. The team must approve these corrections before treating the design as an implementation contract.

6 Peer design review notes

Review source. A rubric-based internal design review was completed for this pre-submission revision. No external reviewing-team names or written classroom notes were present in the supplied files or Canvas history, so none are invented here. The team should replace the source line if actual peer-team notes are available.

6.1 Review action items

ID	Criterion	Issue	Action	Owner	Priority	Due
PR01	Completeness	C5 emits ACK/fault evidence, but the C4 owner has not yet tested how to correlate ACK/fault events with C5 events.	Add a C4 owner review and integration test before the task breakdown.	Kenshi	Must fix	Before task breakdown
PR02	Feasibility	The 20 ms heartbeat and 50 ms command latency budget may be too tight under load.	Prototype the timer-driven loop and record scheduling jitter, p95 latency, and missed-deadline behavior.	Yuanfei	Must fix	Before hardware dependency
PR03	Clarity	AC-01.3 says below-threshold runs are "not evaluated," while D2 proposes evaluated_reps_below_threshold.	Define one evaluation-set vocabulary and update C3 report fields, UI text, and traceability.	Priscilla	Must fix	Before freeze
PR04	Risk	Privacy controls omit SDK temporary files, caches, crash dumps, and debug logs.	Add file, cache, crash-dump, and debug-log requirements to C1 review and deletion verification.	Priscilla	Must fix	Before app-level deletion claim
PR05	Risk	A safe endpoint does not prove the stopping mechanism is physically effective.	Add supervised bench-stop tests before any unsupervised motion.	Yuanfei	Must fix	Before first physical run
PR06	Feasibility	Unknown base-model pretraining exposure could weaken later held-out evaluation.	Log known and unknown model-card/dataset evidence and define how exposure is recorded.	Kenshi	Should fix	Before final evaluation

6.2 Review summary and resulting changes

Most-discussed criterion: Risk. The strongest concerns are physical path safety and complete privacy coverage. Timing feasibility also affects safe fail-closed behavior.

Highest-priority issues.

1. Endpoint checking is not enough: the complete swept and braking path must remain safe.
2. Heartbeat and command-expiry targets must be measured under realistic scheduling and driver load.
3. Persistent event ownership and restart behavior must be explicit so an audit failure cannot silently invalidate a trial.

Changes incorporated into the design plan.

• C4 owns durable session, command, ACK, watchdog, stop, and log-health event records; C5 remains the device boundary and event producer.
• NF03/NF04 require measured latency distributions, not only nominal periods.
• AC-01.3 separates evaluation result from hardware eligibility.
• C1 privacy review covers raw images, shared memory, caches, backups, crash dumps, and derived artifacts.
• G3 closes only after swept-path, stopping-distance, E-stop, and fault-injection evidence exists.

6.3 Issues raised while reviewing other teams

No verifiable notes identifying another reviewed team or project were included in the available records. This section cannot honestly attribute comments to classmates without those notes. The team should insert its two actual classroom observations here before submission if they are available.

Required record format:

1. Reviewed team/project — actual issue — why it mattered — recommendation.
2. Reviewed team/project — second actual issue — why it mattered — recommendation.

6.4 Final review disposition

PR01–PR05 are must-fix before the relevant implementation or hardware milestone. PR06 is worth considering before the final evaluation report. This review improves specificity and traceability but does not authorize hardware testing. G1–G4 and all numerical thresholds still require team validation.

Team validation required

Confirm owners and due dates; replace the review-source note with real peer-team names if available; add the two actual issues raised while reviewing others; approve the evaluation-status correction and C4/C5 event ownership; and keep hardware disabled until timing and full-path safety evidence passes.